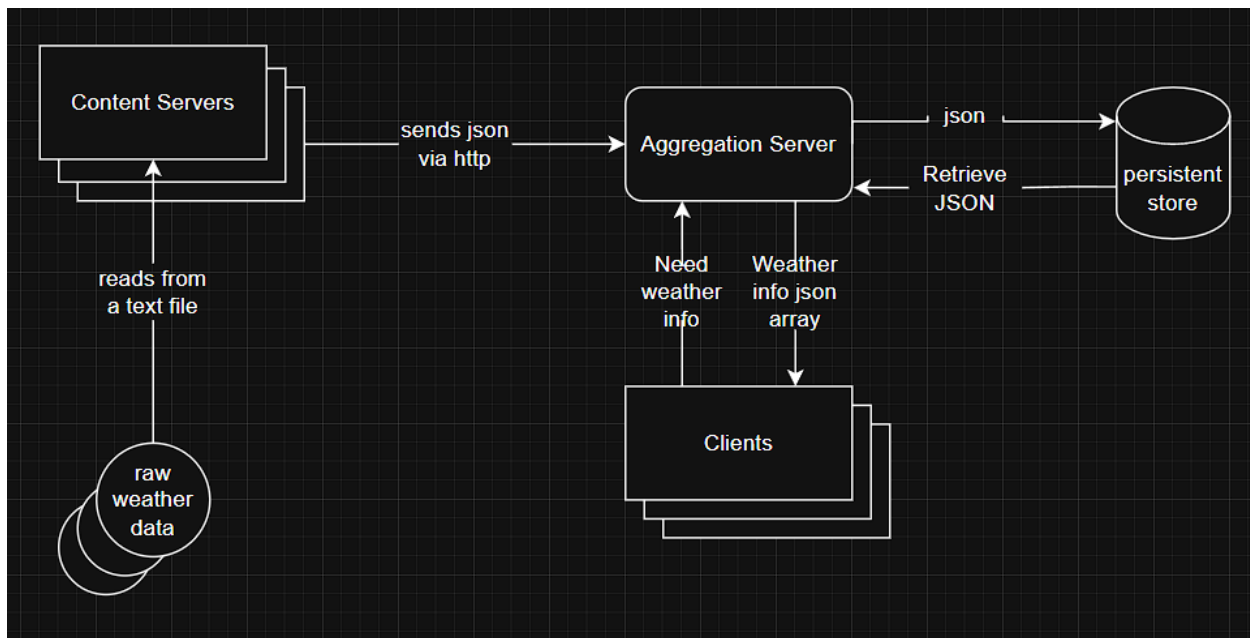


Distributed Systems Assignment 2
Architectural Design Document
A3002558

Here is the design sketch :



Components :

Content Servers: Content Servers are the entity that gather weather data from various source. In case of this case the sources are .txt files. Content servers convert those to Json and send the Json via http to the Aggregation Server.

Aggregation Server: This is the server that aggregates all the weather data it collected and saves it in persistent memory. It also does sanity check of the JSON and serves the client with latest weather data.

Clients: Clients are entity that need weather info. Clients send GET http requests for weather data to Aggregation server. They receive this data via JSON and print it in console.

Raw Weather data: For our project it is some text files.

Persistent Store: For our project it is our local folder.

Distributed Systems Assignment 2

Changes document from lab sessions

A3002558

Proposed changes from lab session:

Initially I wasn't considering multiple content servers. I added a special class for simulating multiple content servers without much hassle in the terminal, *ConcurrentContentServerLauncher*. I also missed implementing fault tolerance from content server's side. Now my content server tries a pre-determined number of times before giving up effort to connect to the Aggregation server.

Distributed Systems Assignment 2

Testing Document

A3002558

1. Types of Testing

Unit Tests

- **Goal:** Verify correctness of individual classes and methods.
- **How:** Each class (LamportClock, ContentServer, AggregationServer, GETClient) has dedicated JUnit test classes.
- **Examples:**
 - LamportClock: Checks clock increment and update.
 - ContentServer: Tests conversion from weather file to JSON.
 - AggregationServer: Checks data storage, expiry logic, and JSON parsing.

Integration Tests

- **Goal:** Ensure parts of the system work together as expected.
- **How:** Integration tests launch AggregationServer, multiple ContentServers, and GETClients to test network interactions and overall data flow.
- **Key Scenarios:**
 - Concurrent GET and PUT requests.
 - Server handles multiple clients at the same time.
 - Data expiry and recovery after server restarts.

Failure Mode and Edge Case Testing

- **Goal:** Confirm system handles errors and unusual conditions gracefully.
- **Examples:**
 - Server unreachable (simulated network error).
 - Malformed or incomplete JSON submissions.
 - Empty PUT requests (204 No Content).
 - Race conditions and thread safety under high concurrency.

2. Test Coverage Criteria

- **Functionality:** Every main feature, such as submitting and retrieving weather data, has related tests.
- **Concurrency:** Multiple clients are run together to check for correct data aggregation and race condition avoidance.
- **Fault Tolerance:** Retry logic for clients, server recovery after crashes, and expiry logic for stale data are all validated.
- **Lamport Clock:** All distributed entities include Lamport clock synchronization in their tests.

3. How to Run Tests

- **Unit Tests:** Run with JUnit in IntelliJ or mvn test if using Maven.
- **Integration Tests:** Use provided launcher or integration test classes. Tests may start the server and clients automatically.
- **Manual Checks:** Run AggregationServer, ContentServer, and GETClient by hand for exploratory testing. Check outputs for expected JSON and error codes.

4. Testing Tools

- **JUnit5:** For writing and running tests.
- **Manual validation:** Observing console outputs and JSON files.
- **Concurrent launchers:** For concurrency and failure simulation.

5. Best Practices Followed

- Each method and class is tested for normal and error behaviors.
- Tests are readable, isolated, and automated for repeatability.
- Thread-safe and concurrent operations are validated under load.
- Testing results documented with clear pass/fail criteria.

For additional details or review of individual test files, see the `/tests` directory and in-code documentation.